Modul Implementasi: Login RBAC, CRUD Role & Pengguna, dan RBAC Berbasis Menu

Project: belajar-laravel (Simulasi Penjadwalan Kuliah)

Stack: Laravel 12 · PHP 8.2 · MySQL · spatie/laravel-permission 6.x

Isi modul (2 bagian):

- **Bagian A — RBAC Control & CRUD Pengguna:** kelola role + permission, dan kelola akun pengguna beserta role-nya.
- **Bagian B — RBAC Berbasis Menu:** tiap menu diwakili permission; menu yang tampil diatur per role dari UI, dan rute yang tidak diizinkan diblokir (403).

Daftar Isi

1. Struktur Tabel RBAC
2. A. RoleController
3. A. View Role
4. A. UserController
5. A. View Pengguna
6. Cara Pakai di UI
7. B. config/menu.php
8. B. RolePermissionSeeder
9. B. routes/web.php
10. B. partials/nav.blade.php
11. B. Model User
12. B. bootstrap/app.php
13. B. AuthController
14. Screenshot Hasil
15. Daftar File Baru
16. Daftar File yang Berubah
17. Verifikasi & Skenario Uji

Struktur Tabel RBAC (Spatie)

Tabel	Fungsi
`roles`	Daftar role (admin, mahasiswa, dosen, ...).
`permissions`	Daftar permission: CRUD (`lihat-data` , ...) dan menu (`menu-*`).
`model_has_roles`	Relasi user ↔ role.
`model_has_permissions`	Relasi user ↔ permission (langsung).
`role_has_permissions`	Relasi role ↔ permission — diubah lewat menu RBAC Control.

Bagian A — RBAC Control & CRUD Pengguna

A. RoleController

File: `app/Http/Controllers/RoleController.php` BARU

CRUD role sekaligus menetapkan permission ke role (`role_has_permissions`). Menu RBAC Control punya dua tampilan menu sekaligus: daftar role (dengan permission & jumlah pengguna) dan form tambah/ubah role berisi checkbox permission.

```
<?php

namespace App\Http\Controllers;

use Illuminate\Http\RedirectResponse;
use Illuminate\Http\Request;
use Illuminate\Validation\Rule;
use Illuminate\View\View;
use Spatie\Permission\Models\Permission;
use Spatie\Permission\Models\Role;
use Spatie\Permission\PermissionRegistrar;

class RoleController extends Controller
{
    public function index(): View
    {
        $roles = Role::withCount('users')->with('permissions')->orderBy('id')->get();

        return view('role.index', compact('roles'));
    }

    public function create(): View
    {
        $permissions = Permission::orderBy('name')->get();

        return view('role.create', compact('permissions'));
    }

    public function store(Request $request): RedirectResponse
    {
        $validated = $request->validate([
```

```
            'name' => ['required', 'string', 'max:50', Rule::unique('roles', 'name')]
            'permissions' => ['nullable', 'array'],
            'permissions.*' => ['exists:permissions,name'],
        ]);

        $role = Role::create(['name' => $validated['name']]);
        $role->syncPermissions($validated['permissions'] ?? []);

        app(PermissionRegistrar::class)->forgetCachedPermissions();

        return redirect()
            ->route('role.index')
            ->with('success', 'Data role berhasil ditambahkan.');
    }

    public function edit(Role $role): View
    {
        $permissions = Permission::orderBy('name')->get();

        return view('role.edit', compact('role', 'permissions'));
    }

    public function update(Request $request, Role $role): RedirectResponse
    {
        $validated = $request->validate([
            'name' => ['required', 'string', 'max:50', Rule::unique('roles', 'name')-
            'permissions' => ['nullable', 'array'],
            'permissions.*' => ['exists:permissions,name'],
        ]);

        $role->update(['name' => $validated['name']]);
        $role->syncPermissions($validated['permissions'] ?? []);

        app(PermissionRegistrar::class)->forgetCachedPermissions();

        return redirect()
            ->route('role.index')
            ->with('success', 'Data role berhasil diubah.');
    }

    public function destroy(Role $role): RedirectResponse
    {
        if ($role->users()->exists()) {
```

```
            return redirect()
                ->route('role.index')
                ->with('error', 'Role tidak bisa dihapus karena masih dipakai oleh pe
        }

        $role->delete();

        app(PermissionRegistrar::class)->forgetCachedPermissions();

        return redirect()
            ->route('role.index')
            ->with('success', 'Data role berhasil dihapus.');
    }
}
```

- `withCount('users') + with('permissions')` agar daftar role tidak N+1.
- `syncPermissions()` menyinkronkan centang permission; `forgetCachedPermissions()` mereset cache Spatie.
- `destroy()` menolak menghapus role yang masih dipakai pengguna (flash `error`).

A. View Role

File: `resources/views/role/index.blade.php` BARU

```
<!DOCTYPE html>
<html lang="id">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Data Role</title>
    <style>
        @include('partials.style')
    </style>
</head>
<body>
    <h1>Data Role</h1>

    @if (session('success'))
        <p class="success">{{ session('success') }}</p>
    @endif
```

```
@if (session('error'))
    <ul class="errors">
        <li>{{ session('error') }}</li>
    </ul>
@endif

@include('partials.nav')

@role('admin')
    <p>
        <a href="{{ route('role.create') }}" class="btn">Tambah Role</a>
    </p>
@endrole

<table>
    <thead>
        <tr>
            <th>No</th>
            <th>Nama Role</th>
            <th>Permission</th>
            <th>Jumlah Pengguna</th>
            <th>Action</th>
        </tr>
    </thead>
    <tbody>
        @forelse ($roles as $role)
            <tr>
                <td>{{ $loop->iteration }}</td>
                <td>{{ $role->name }}</td>
                <td>{{ $role->permissions->pluck('name')->implode(', ') ?: '-' }}
                <td>{{ $role->users_count }}</td>
                <td>
                    @role('admin')
                        <a href="{{ route('role.edit', $role->id) }}" class="btn"

                        <form
                            action="{{ route('role.destroy', $role->id) }}"
                            method="POST"
                            style="display: inline"
                            onsubmit="return confirm('Yakin ingin menghapus role
                        >
                            @csrf
                            @method('DELETE')
```

```
                                <button type="submit">Hapus</button>
                            </form>
                        @else
                            <span>-</span>
                        @endrole
                    </td>
                </tr>
            @empty
                <tr>
                    <td colspan="5">Belum ada data role.</td>
                </tr>
            @endforelse
        </tbody>
    </table>
</body>
</html>
```

File: `resources/views/role/create.blade.php` BARU

```
<!DOCTYPE html>
<html lang="id">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Tambah Role</title>
    <style>
        @include('partials.style')
        body {
            max-width: 600px;
        }
    </style>
</head>
<body>
    <h1>Tambah Role</h1>

    @if ($errors->any())
        <ul class="errors">
            @foreach ($errors->all() as $error)
                <li>{{ $error }}</li>
            @endforeach
        </ul>
    @endif
```

```
    <form action="{{ route('role.store') }}" method="POST">
        @csrf

        <p>
            <label for="name">Nama Role</label><br>
            <input type="text" id="name" name="name" value="{{ old('name') }}" maxlen
        </p>

        <p>
            <label>Permission</label>
        </p>

        @foreach ($permissions as $permission)
            <p>
                <label style="font-weight: normal;">
                    <input type="checkbox" name="permissions[]" value="{{ $permission
                    {{ $permission->name }}
                </label>
            </p>
        @endforeach

        <button type="submit">Simpan</button>
        <a href="{{ route('role.index') }}" class="btn">Batal</a>
    </form>
</body>
</html>
```

File: `resources/views/role/edit.blade.php` BARU

```
<!DOCTYPE html>
<html lang="id">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Ubah Data Role</title>
    <style>
        @include('partials.style')
        body {
            max-width: 600px;
        }
    </style>
</head>
```

```
<body>
    <h1>Ubah Data Role</h1>

    @if ($errors->any())
        <ul class="errors">
            @foreach ($errors->all() as $error)
                <li>{{ $error }}</li>
            @endforeach
        </ul>
    @endif

    <form action="{{ route('role.update', $role->id) }}" method="POST">
        @csrf
        @method('PUT')

        <p>
            <label for="name">Nama Role</label><br>
            <input type="text" id="name" name="name" value="{{ old('name', $role->nam
        </p>

        <p>
            <label>Permission</label>
        </p>

        @foreach ($permissions as $permission)
            <p>
                <label style="font-weight: normal;">
                    <input type="checkbox" name="permissions[]" value="{{ $permission
                    {{ $permission->name }}
                </label>
            </p>
        @endforeach

        <button type="submit">Simpan Perubahan</button>
        <a href="{{ route('role.index') }}" class="btn">Batal</a>
    </form>
</body>
</html>
```

A. UserController

File: `app/Http/Controllers/UserController.php` BARU

CRUD akun pengguna: nama, email, password, dan satu role (dropdown). Password otomatis di-hash oleh cast `'password' => 'hashed'` pada model `User`.

```
<?php

namespace App\Http\Controllers;

use App\Models\User;
use Illuminate\Http\RedirectResponse;
use Illuminate\Http\Request;
use Illuminate\Validation\Rule;
use Illuminate\View\View;
use Spatie\Permission\Models\Role;

class UserController extends Controller
{
    public function index(): View
    {
        $users = User::with('roles')->orderBy('id')->get();

        return view('pengguna.index', compact('users'));
    }

    public function create(): View
    {
        $roles = Role::orderBy('name')->get();

        return view('pengguna.create', compact('roles'));
    }

    public function store(Request $request): RedirectResponse
    {
        $validated = $request->validate([
            'name' => ['required', 'string', 'max:100'],
            'email' => ['required', 'email', 'max:100', Rule::unique('users', 'email'
            'password' => ['required', 'string', 'min:8'],
            'role' => ['required', 'exists:roles,name'],
        ]);

        $user = User::create([
            'name' => $validated['name'],
```

```
            'email' => $validated['email'],
            'password' => $validated['password'],
        ]);
        $user->syncRoles([$validated['role']]);

        return redirect()
            ->route('pengguna.index')
            ->with('success', 'Data pengguna berhasil ditambahkan.');
    }

    public function edit(User $pengguna): View
    {
        $roles = Role::orderBy('name')->get();

        return view('pengguna.edit', compact('pengguna', 'roles'));
    }

    public function update(Request $request, User $pengguna): RedirectResponse
    {
        $validated = $request->validate([
            'name' => ['required', 'string', 'max:100'],
            'email' => ['required', 'email', 'max:100', Rule::unique('users', 'email'
            'password' => ['nullable', 'string', 'min:8'],
            'role' => ['required', 'exists:roles,name'],
        ]);

        $pengguna->name = $validated['name'];
        $pengguna->email = $validated['email'];

        if (! empty($validated['password'])) {
            $pengguna->password = $validated['password'];
        }

        $pengguna->save();
        $pengguna->syncRoles([$validated['role']]);

        return redirect()
            ->route('pengguna.index')
            ->with('success', 'Data pengguna berhasil diubah.');
    }

    public function destroy(User $pengguna): RedirectResponse
    {
```

```
        if ($pengguna->is(auth()->user())) {
            return redirect()
                ->route('pengguna.index')
                ->with('error', 'Anda tidak bisa menghapus akun yang sedang digunakan
        }

        $pengguna->delete();

        return redirect()
            ->route('pengguna.index')
            ->with('success', 'Data pengguna berhasil dihapus.');
    }
}
```

- Password `nullable` saat update: kosong = tidak diubah.
- `destroy()` menolak menghapus akun yang sedang login.

A. View Pengguna

File: `resources/views/pengguna/index.blade.php` BARU

```
<!DOCTYPE html>
<html lang="id">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Data Pengguna</title>
    <style>
        @include('partials.style')
    </style>
</head>
<body>
    <h1>Data Pengguna</h1>

    @if (session('success'))
        <p class="success">{{ session('success') }}</p>
    @endif

    @if (session('error'))
        <ul class="errors">
            <li>{{ session('error') }}</li>
```

```
    </ul>
@endif

@include('partials.nav')

@role('admin')
    <p>
        <a href="{{ route('pengguna.create') }}" class="btn">Tambah Pengguna</a>
    </p>
@endrole

<table>
    <thead>
        <tr>
            <th>No</th>
            <th>Nama</th>
            <th>Email</th>
            <th>Role</th>
            <th>Action</th>
        </tr>
    </thead>
    <tbody>
        @forelse ($users as $user)
            <tr>
                <td>{{ $loop->iteration }}</td>
                <td>{{ $user->name }}</td>
                <td>{{ $user->email }}</td>
                <td>{{ $user->getRoleNames()->implode(', ') ?: '-' }}</td>
                <td>
                    @role('admin')
                        <a href="{{ route('pengguna.edit', $user->id) }}" class="

                        <form
                            action="{{ route('pengguna.destroy', $user->id) }}"
                            method="POST"
                            style="display: inline"
                            onsubmit="return confirm('Yakin ingin menghapus pengg
                        >
                            @csrf
                            @method('DELETE')
                            <button type="submit">Hapus</button>
                        </form>
                    @else
```

```
                            <span>-</span>
                        @endrole
                    </td>
                </tr>
            @empty
                <tr>
                    <td colspan="5">Belum ada data pengguna.</td>
                </tr>
            @endforelse
        </tbody>
    </table>
</body>
</html>
```

File: `resources/views/pengguna/create.blade.php` BARU

```
<!DOCTYPE html>
<html lang="id">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Tambah Pengguna</title>
    <style>
        @include('partials.style')
        body {
            max-width: 600px;
        }
    </style>
</head>
<body>
    <h1>Tambah Pengguna</h1>

    @if ($errors->any())
        <ul class="errors">
            @foreach ($errors->all() as $error)
                <li>{{ $error }}</li>
            @endforeach
        </ul>
    @endif

    <form action="{{ route('pengguna.store') }}" method="POST">
        @csrf
```

```
        <p>
            <label for="name">Nama</label><br>
            <input type="text" id="name" name="name" value="{{ old('name') }}" maxlen
        </p>

        <p>
            <label for="email">Email</label><br>
            <input type="email" id="email" name="email" value="{{ old('email') }}" ma
        </p>

        <p>
            <label for="password">Password</label><br>
            <input type="password" id="password" name="password" minlength="8" requir
        </p>

        <p>
            <label for="role">Role</label><br>
            <select id="role" name="role" required>
                <option value="">-- Pilih Role --</option>
                @foreach ($roles as $r)
                    <option value="{{ $r->name }}" {{ old('role') == $r->name ? 'sele
                        {{ $r->name }}
                    </option>
                @endforeach
            </select>
        </p>

        <button type="submit">Simpan</button>
        <a href="{{ route('pengguna.index') }}" class="btn">Batal</a>
    </form>
</body>
</html>
```

File: `resources/views/pengguna/edit.blade.php` BARU

```
<!DOCTYPE html>
<html lang="id">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Ubah Data Pengguna</title>
    <style>
        @include('partials.style')
```

```
        body {
            max-width: 600px;
        }
    </style>
</head>
<body>
    <h1>Ubah Data Pengguna</h1>

    @if ($errors->any())
        <ul class="errors">
            @foreach ($errors->all() as $error)
                <li>{{ $error }}</li>
            @endforeach
        </ul>
    @endif

    <form action="{{ route('pengguna.update', $pengguna->id) }}" method="POST">
        @csrf
        @method('PUT')

        <p>
            <label for="name">Nama</label><br>
            <input type="text" id="name" name="name" value="{{ old('name', $pengguna-
        </p>

        <p>
            <label for="email">Email</label><br>
            <input type="email" id="email" name="email" value="{{ old('email', $pengg
        </p>

        <p>
            <label for="password">Password</label><br>
            <input type="password" id="password" name="password" minlength="8">
            <small>Kosongkan jika tidak ingin mengubah password.</small>
        </p>

        <p>
            <label for="role">Role</label><br>
            <select id="role" name="role" required>
                <option value="">-- Pilih Role --</option>
                @foreach ($roles as $r)
                    <option value="{{ $r->name }}" {{ old('role', $pengguna->getRoleN
                        {{ $r->name }}
```

```
                    </option>
                @endforeach
            </select>
        </p>

        <button type="submit">Simpan Perubahan</button>
        <a href="{{ route('pengguna.index') }}" class="btn">Batal</a>
    </form>
</body>
</html>
```

Cara Pakai di UI

1. Login sebagai admin.
2. Menu **RBAC Control** → **Tambah Role** (mis. `dosen`) → centang menu/permission yang boleh.
3. Menu **Pengguna** → **Tambah Pengguna** → isi nama/email/password → pilih role.
4. Login sebagai user tersebut → hanya menu yang diizinkan yang tampil.

Bagian B — RBAC Berbasis Menu

B. config/menu.php

File: `config/menu.php` BARU

Pemetaan menu → permission → route. Dipakai bersama oleh nav (menampilkan) dan `User::firstAccessibleMenuUrl()` (menentukan halaman tujuan setelah login). Urutan array = urutan menu sekaligus prioritas redirect.

```
<?php

return [
    'mahasiswa'     => ['label' => 'Mahasiswa',     'permission' => 'menu-mahasiswa',
    'dosen'         => ['label' => 'Dosen',         'permission' => 'menu-dosen',
    'mata-kuliah'   => ['label' => 'Mata Kuliah',   'permission' => 'menu-mata-kuliah
    'ruangan'       => ['label' => 'Ruangan',       'permission' => 'menu-ruangan',
    'kelas'         => ['label' => 'Kelas',         'permission' => 'menu-kelas',
```

```
    'jadwal-kuliah' => ['label' => 'Jadwal Kuliah', 'permission' => 'menu-jadwal-kuli
];
```

B. RolePermissionSeeder

File: `database/seeders/RolePermissionSeeder.php` BERUBAH

Menambah permission menu dan meng-assign-nya: role `admin` mendapat semua menu, role `mahasiswa` mendapat `menu-jadwal-kuliah` saja (contoh). Idempoten, aman dijalankan ulang.

```
<?php

namespace Database\Seeders;

use App\Models\User;
use Illuminate\Database\Seeder;
use Spatie\Permission\Models\Permission;
use Spatie\Permission\Models\Role;
use Spatie\Permission\PermissionRegistrar;

class RolePermissionSeeder extends Seeder
{
    /**
     * Jalankan seeder role, permission, dan user default.
     */
    public function run(): void
    {
        // Reset cache permission agar data terbaru langsung terbaca.
        app(PermissionRegistrar::class)->forgetCachedPermissions();

        // 1. Buat permission dasar + permission menu (dipakai untuk RBAC per menu).
        $permissions = ['lihat-data', 'tambah-data', 'ubah-data', 'hapus-data'];

        $menuPermissions = [
            'menu-mahasiswa', 'menu-dosen', 'menu-mata-kuliah',
            'menu-ruangan', 'menu-kelas', 'menu-jadwal-kuliah',
        ];

        foreach (array_merge($permissions, $menuPermissions) as $permission) {
            Permission::firstOrCreate(['name' => $permission]);
        }
```

```
        // 2. Buat role dan assigns permission.
        // Role admin: semua permission + semua menu.
        $roleAdmin = Role::firstOrCreate(['name' => 'admin']);
        $roleAdmin->syncPermissions(array_merge($permissions, $menuPermissions));

        // Role mahasiswa: read-only + hanya menu Jadwal Kuliah (contoh, bisa diubah
        $roleMahasiswa = Role::firstOrCreate(['name' => 'mahasiswa']);
        $roleMahasiswa->syncPermissions(['lihat-data', 'menu-jadwal-kuliah']);

        // 3. Buat user default (password otomatis di-hash oleh cast 'hashed' pada mo
        $admin = User::firstOrCreate(
            ['email' => 'admin@example.test'],
            ['name' => 'Administrator', 'password' => 'password']
        );
        $admin->syncRoles(['admin']);

        $mahasiswa = User::firstOrCreate(
            ['email' => 'mahasiswa@example.test'],
            ['name' => 'Mahasiswa Demo', 'password' => 'password']
        );
        $mahasiswa->syncRoles(['mahasiswa']);

        // Reset cache kembali setelah data tersimpan.
        app(PermissionRegistrar::class)->forgetCachedPermissions();
    }
}
```

B. routes/web.php

File: `routes/web.php` BERUBAH

Versi final: tiap menu punya grup `permission:menu-<slug>`. Baca (index/show) cukup permission menu; tulis (create/store/edit/update/destroy) tambahan `role:admin`. Resource `role` & `pengguna` tetap khusus admin.

```
<?php

use Illuminate\Support\Facades\Route;
use App\Http\Controllers\AuthController;
use App\Http\Controllers\DosenController;
```

```
use App\Http\Controllers\JadwalKuliahController;
use App\Http\Controllers\KelasController;
use App\Http\Controllers\MahasiswaController;
use App\Http\Controllers\MataKuliahController;
use App\Http\Controllers\RoleController;
use App\Http\Controllers\RuanganController;
use App\Http\Controllers\UserController;

Route::get('/', function () {
    return view('welcome');
});

/*
|--------------------------------------------------------------------------
| Route Autentikasi
|--------------------------------------------------------------------------
| Halaman login hanya untuk tamu (guest). Logout butuh status login.
*/
Route::middleware('guest')->group(function () {
    Route::get('/login', [AuthController::class, 'showLoginForm'])->name('login');
    Route::post('/login', [AuthController::class, 'login']);
});

Route::post('/logout', [AuthController::class, 'logout'])
    ->middleware('auth')
    ->name('logout');

/*
|--------------------------------------------------------------------------
| Route Menu Konten: dijaga permission "menu-<slug>"
|--------------------------------------------------------------------------
| Menu yang boleh tampil diatur per role lewat permission menu (RBAC Control).
| Aksi tulis (create/store/edit/update/destroy) tetap khusus role admin.
| Grup tulis didaftarkan LEBIH DULU agar "create" tidak tertangkap "{id}".
*/
Route::middleware('auth')->group(function () {
    Route::middleware('permission:menu-mahasiswa')->group(function () {
        Route::resource('mahasiswa', MahasiswaController::class)->except(['index', 's
        Route::resource('mahasiswa', MahasiswaController::class)->only(['index', 'sho
    });

    Route::middleware('permission:menu-dosen')->group(function () {
        Route::resource('dosen', DosenController::class)->except(['index', 'show'])->
```

```
        Route::resource('dosen', DosenController::class)->only(['index', 'show']);
    });

    Route::middleware('permission:menu-mata-kuliah')->group(function () {
        Route::resource('mata-kuliah', MataKuliahController::class)->except(['index',
        Route::resource('mata-kuliah', MataKuliahController::class)->only(['index', '
    });

    Route::middleware('permission:menu-ruangan')->group(function () {
        Route::resource('ruangan', RuanganController::class)->except(['index', 'show'
        Route::resource('ruangan', RuanganController::class)->only(['index', 'show'])
    });

    Route::middleware('permission:menu-kelas')->group(function () {
        Route::resource('kelas', KelasController::class)->except(['index', 'show'])->
            'kelas' => 'kls',
        ]);
        Route::resource('kelas', KelasController::class)->only(['index', 'show'])->pa
            'kelas' => 'kls',
        ]);
    });

    Route::middleware('permission:menu-jadwal-kuliah')->group(function () {
        Route::resource('jadwal-kuliah', JadwalKuliahController::class)->except(['ind
        Route::resource('jadwal-kuliah', JadwalKuliahController::class)->only(['index
    });

    /*
    |--------------------------------------------------------------------------
    | Manajemen RBAC (role) & pengguna: selalu khusus admin (pengaman)
    |--------------------------------------------------------------------------
    */
    Route::middleware('role:admin')->group(function () {
        Route::resource('role', RoleController::class)->except('show');
        Route::resource('pengguna', UserController::class)->except('show');
    });
});
```

B. partials/nav.blade.php

File: `resources/views/partials/nav.blade.php` BERUBAH

Versi final: menu konten dirender dari `config('menu')` dan hanya tampil bila user punya permission-nya (`@can`). Menu Role & Pengguna tetap `@role('admin')` .

```
<nav class="main-nav">
    <div class="nav-links">
        @foreach (config('menu') as $menu)
            @can($menu['permission'])
                <a href="{{ route($menu['route']) }}">{{ $menu['label'] }}</a>
            @endcan
        @endforeach

        @role('admin')
            <a href="{{ route('role.index') }}">Role</a>
            <a href="{{ route('pengguna.index') }}">Pengguna</a>
        @endrole
    </div>

    @auth
        <div class="nav-user">
            <span class="user-info">
                {{ auth()->user()->name }}
                ({{ auth()->user()->getRoleNames()->implode(', ') }})
            </span>

            <form method="POST" action="{{ route('logout') }}">
                @csrf
                <button type="submit">Logout</button>
            </form>
        </div>
    @endauth
</nav>
```

B. Model User

File: `app/Models/User.php` BERUBAH

Menambah helper `firstAccessibleMenuUrl()` untuk menentukan menu pertama yang boleh diakses (redirect setelah login), supaya role tanpa `menu-mahasiswa` tidak mendarat di 403.

```php
<?php

namespace App\Models;

// use Illuminate\Contracts\Auth\MustVerifyEmail;
use Database\Factories\UserFactory;
use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Foundation\Auth\User as Authenticatable;
use Illuminate\Notifications\Notifiable;
use Spatie\Permission\Traits\HasRoles;

class User extends Authenticatable
{
    /** @use HasFactory<UserFactory> */
    use HasFactory, Notifiable, HasRoles;

    /**
     * The attributes that are mass assignable.
     *
     * @var list<string>
     */
    protected $fillable = [
        'name',
        'email',
        'password',
    ];

    /**
     * The attributes that should be hidden for serialization.
     *
     * @var list<string>
     */
    protected $hidden = [
        'password',
        'remember_token',
    ];

    /**
     * Get the attributes that should be cast.
     *
     * @return array<string, string>
     */
    protected function casts(): array
```

```
    {
        return [
            'email_verified_at' => 'datetime',
            'password' => 'hashed',
        ];
    }

    /**
     * URL menu pertama yang boleh diakses user (berdasarkan permission menu).
     */
    public function firstAccessibleMenuUrl(): string
    {
        foreach (config('menu') as $menu) {
            if ($this->can($menu['permission'])) {
                return route($menu['route']);
            }
        }

        return '/';
    }
}
```

B. bootstrap/app.php

File: `bootstrap/app.php` BERUBAH

```
<?php

use Illuminate\Foundation\Application;
use Illuminate\Foundation\Configuration\Exceptions;
use Illuminate\Foundation\Configuration\Middleware;

return Application::configure(basePath: dirname(__DIR__))
    ->withRouting(
        web: __DIR__.'/../routes/web.php',
        commands: __DIR__.'/../routes/console.php',
        health: '/up',
    )
    ->withMiddleware(function (Middleware $middleware): void {
        // Arahkan user yang sudah login ke menu pertama yang boleh diaksesnya.
        $middleware->redirectUsersTo(
```

```
            fn () => auth()->user()?->firstAccessibleMenuUrl() ?? route('login')
        );

        $middleware->alias([
            'role' => \Spatie\Permission\Middleware\RoleMiddleware::class,
            'permission' => \Spatie\Permission\Middleware\PermissionMiddleware::class
            'role_or_permission' => \Spatie\Permission\Middleware\RoleOrPermissionMid
        ]);
    })
    ->withExceptions(function (Exceptions $exceptions): void {
        //
    })->create();
```

B. AuthController

File: `app/Http/Controllers/AuthController.php` BERUBAH

Setelah login berhasil, redirect ke menu pertama yang boleh diakses.

```
<?php

namespace App\Http\Controllers;

use Illuminate\Http\RedirectResponse;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Auth;
use Illuminate\Validation\ValidationException;
use Illuminate\View\View;

class AuthController extends Controller
{
    /**
     * Tampilkan form login.
     */
    public function showLoginForm(): View
    {
        return view('auth.login');
    }

    /**
     * Proses login.
     */
```

```
    public function login(Request $request): RedirectResponse
    {
        $credentials = $request->validate([
            'email' => ['required', 'email'],
            'password' => ['required'],
        ]);

        if (Auth::attempt($credentials, $request->boolean('remember'))) {
            $request->session()->regenerate();

            return redirect()->intended(auth()->user()->firstAccessibleMenuUrl());
        }

        throw ValidationException::withMessages([
            'email' => ['Email atau password salah.'],
        ]);
    }

    /**
     * Proses logout.
     */
    public function logout(Request $request): RedirectResponse
    {
        Auth::logout();

        $request->session()->invalidate();
        $request->session()->regenerateToken();

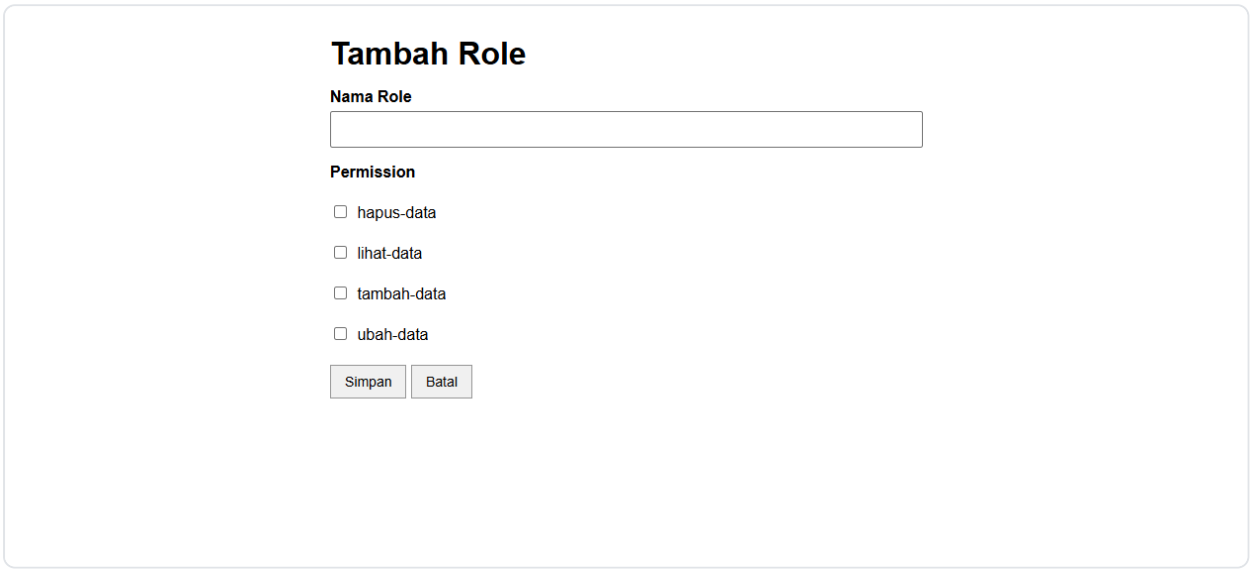
        return redirect()->route('login');
    }
}
```

Screenshot Hasil

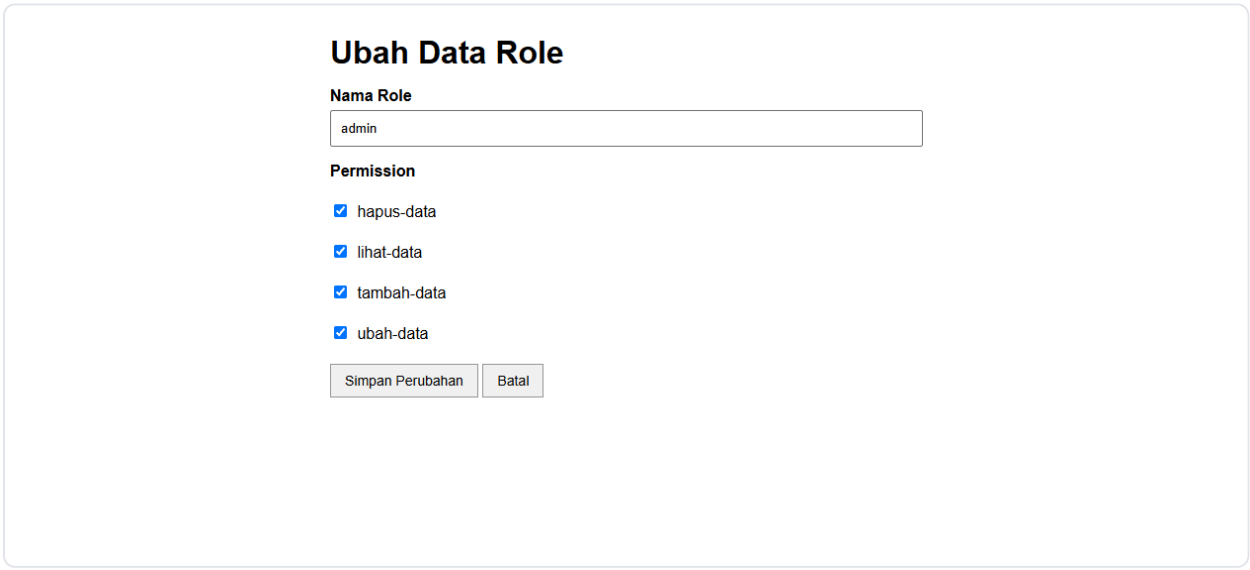
Bagian A — RBAC Control & CRUD Pengguna



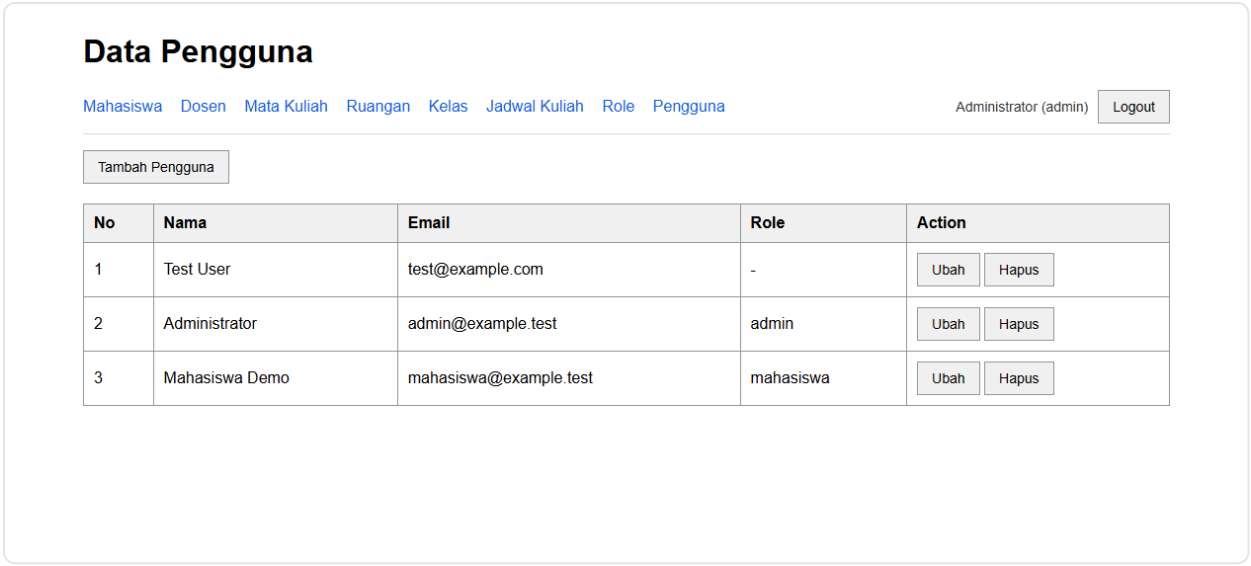
Halaman `/role` — daftar role, permission yang dimiliki, dan jumlah pengguna.



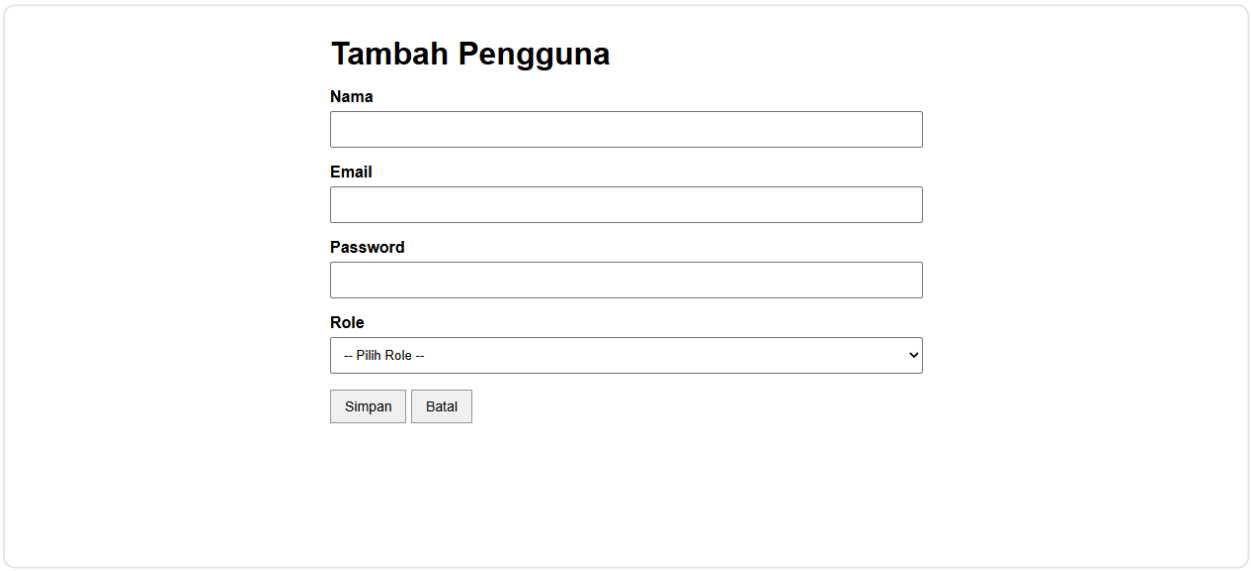
Halaman `/role/create` — nama role + checkbox permission.



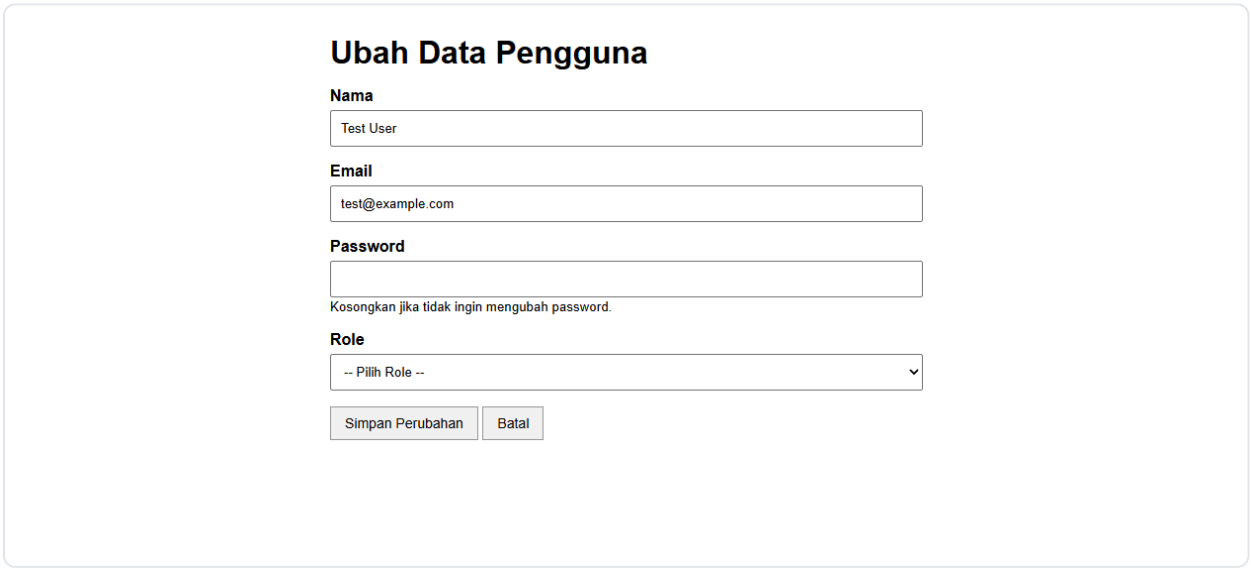
Halaman `/role/{id}/edit` — permission yang sudah dimiliki tercentang otomatis.



Halaman `/pengguna` — daftar akun beserta role-nya.

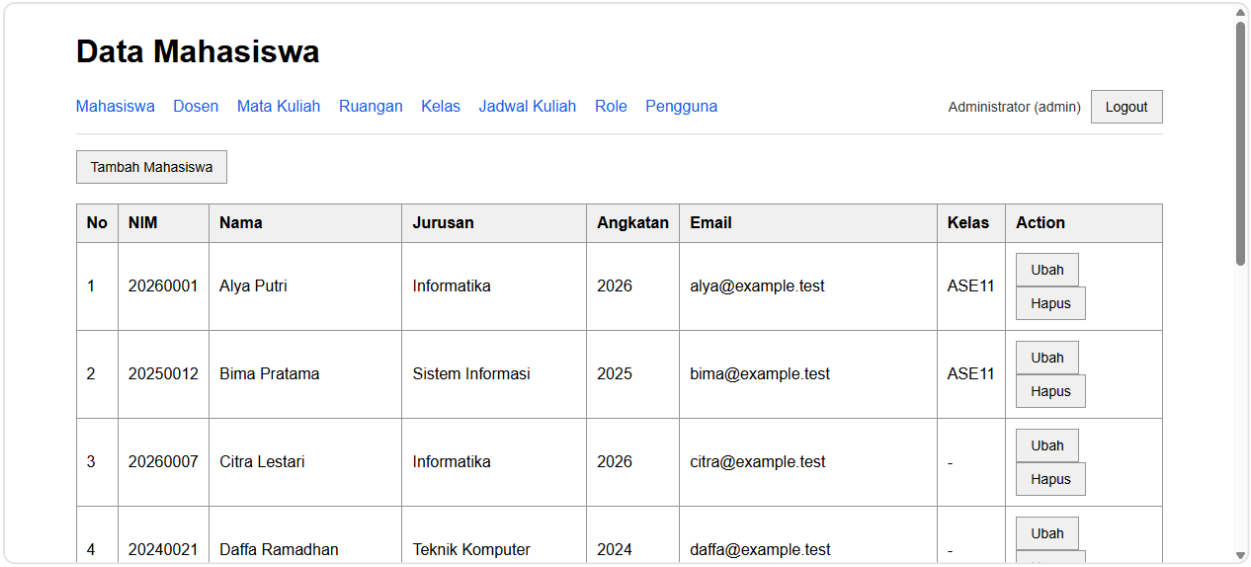


Halaman `/pengguna/create` — nama, email, password, dan pilihan role.

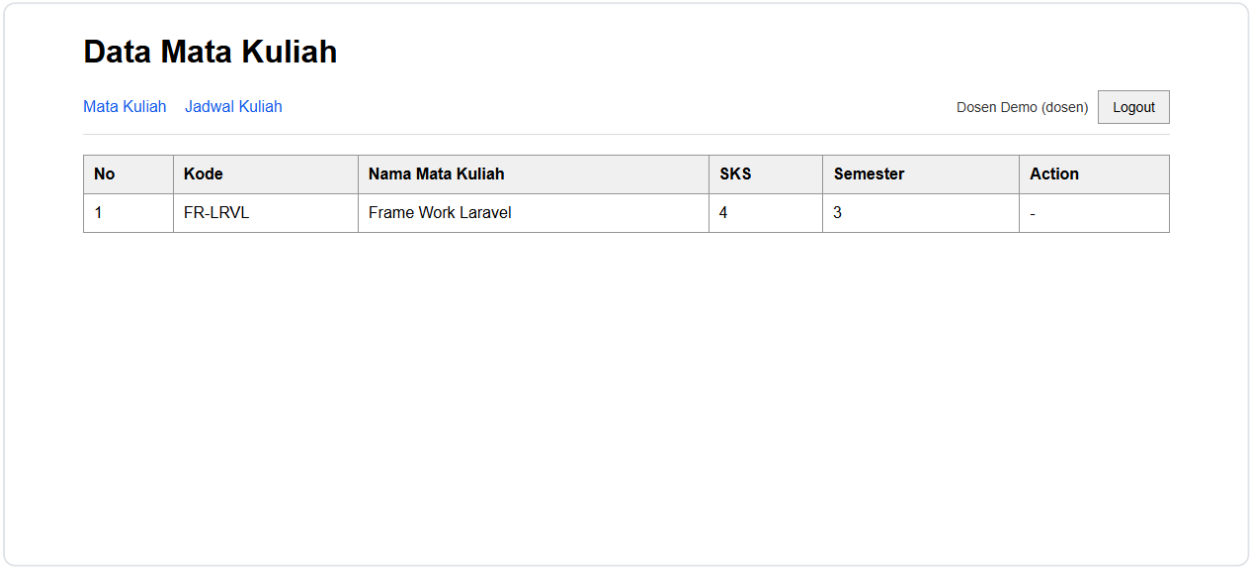


Halaman `/pengguna/{id}/edit` — password opsional (kosong = tidak diubah).

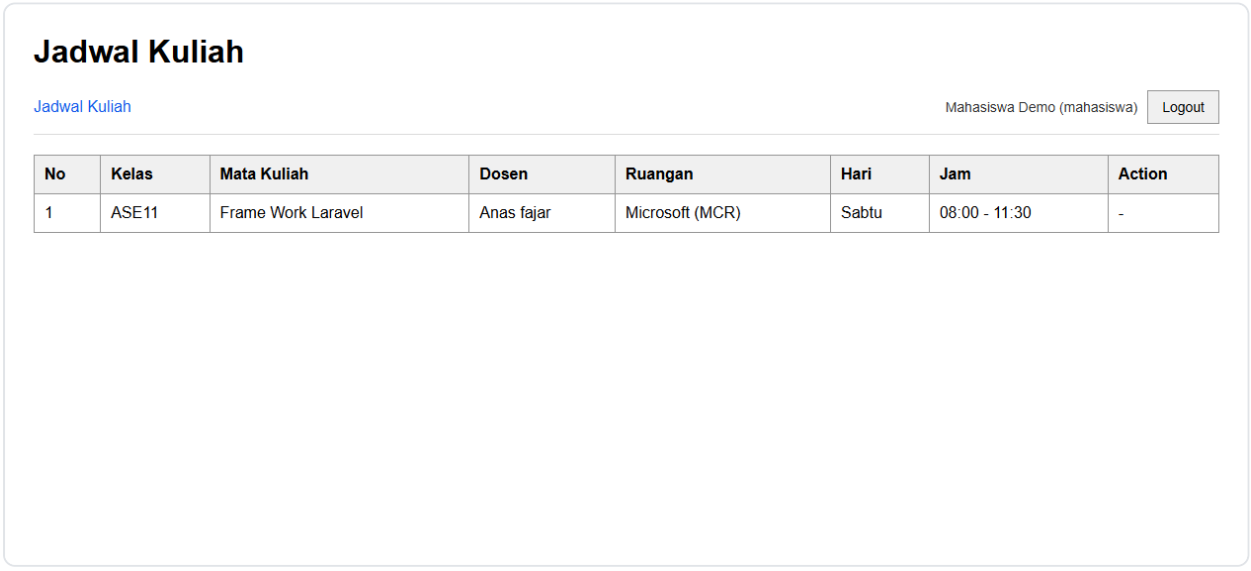
Bagian B — RBAC Berbasis Menu



Admin — semua menu konten + Role & Pengguna tampil.



Role **dosen** — hanya menu Mata Kuliah & Jadwal Kuliah.



Role **mahasiswa** — hanya menu Jadwal Kuliah.

Akses langsung ke menu di luar izin (mis. `/mahasiswa` oleh dosen) — **403 Forbidden**.

Daftar File Baru

File	Keterangan
`app/Http/Controllers/RoleController.php`	CRUD role + assign permission.
`app/Http/Controllers/UserController.php`	CRUD pengguna + pilih satu role.
`resources/views/role/index.blade.php`	Daftar role.
`resources/views/role/create.blade.php`	Form tambah role + permission.
`resources/views/role/edit.blade.php`	Form ubah role + permission.
`resources/views/pengguna/index.blade.php`	Daftar pengguna.
`resources/views/pengguna/create.blade.php`	Form tambah pengguna.
`resources/views/pengguna/edit.blade.php`	Form ubah pengguna.
`config/menu.php`	Pemetaan menu → permission → route.
`screenshots/role-index.png`	Screenshot daftar role.
`screenshots/role-create.png`	Screenshot form tambah role.
`screenshots/role-edit.png`	Screenshot form ubah role.
`screenshots/pengguna-index.png`	Screenshot daftar pengguna.
`screenshots/pengguna-create.png`	Screenshot form tambah pengguna.
`screenshots/pengguna-edit.png`	Screenshot form ubah pengguna.

File	Keterangan
`screenshots/nav-admin-full.png`	Screenshot navigasi admin (semua menu).
`screenshots/nav-dosen.png`	Screenshot navigasi role dosen (2 menu).
`screenshots/nav-mahasiswa.png`	Screenshot navigasi role mahasiswa (1 menu).
`screenshots/forbidden-dosen.png`	Screenshot halaman 403.
`MODUL_RBAC.html`	Modul implementasi ini.

Daftar File yang Berubah

File	Perubahan
`routes/web.php`	Resource `role` & `pengguna` (Bagian A); lalu tiap menu dijaga `permission:menu-<slug>`, aksi tulis tambah `role:admin` (Bagian B).
`resources/views/partials/nav.blade.php`	Tautan Role & Pengguna (Bagian A); lalu menu dirender dari `config('menu')` dengan `@can` (Bagian B).
`database/seeders/RolePermissionSeeder.php`	Tambah permission `menu-*`; admin = semua menu, mahasiswa = `menu-jadwal-kuliah`.
`app/Models/User.php`	Tambah `firstAccessibleMenuUrl()`.
`bootstrap/app.php`	`redirectUsersTo` memakai menu pertama yang boleh diakses.
`app/Http/Controllers/AuthController.php`	Redirect setelah login memakai `firstAccessibleMenuUrl()`.

Verifikasi & Skenario Uji

Bagian A

- Route `role` & `pengguna`: 6 rute masing-masing, semua `auth, role:admin`.
- Tambah/hapus role + centang permission; hapus role yang dipakai user → ditolak.

- Tambah pengguna + role; ubah tanpa isi password → password lama tetap berlaku; hapus akun sendiri → ditolak.

Bagian B

- Permission tersimpan: admin = 4 CRUD + 6 `menu-*` ; mahasiswa = `lihat-data` + `menu-jadwal-kuliah` .
- Route `mahasiswa` : index/show `auth, permission:menu-mahasiswa` ; tulis `auth, permission:menu-mahasiswa, role:admin` .
- Admin → nav semua menu; buat role *dosen* (Mata Kuliah + Jadwal Kuliah) → login dosen hanya 2 menu, `/mahasiswa` → 403; mahasiswa hanya 1 menu, `/dosen` → 403; tiap user mendarat di menu pertamanya.

Langkah menerapkan pada DB yang sudah ada

```
php artisan db:seed --class=RolePermissionSeeder
php artisan optimize:clear
```

Menambah menu baru nanti: tambah 1 baris di `config/menu.php` , tambah permission `menu-*` di seeder, dan tambah grup route-nya. Role tinggal dicentang dari UI RBAC Control.

Catatan: aksi tulis (tambah/ubah/hapus) tetap khusus admin. Bisa diperluas nanti dengan permission CRUD per-menu (mis. `ubah-mahasiswa`) bila role lain perlu ikut mengedit.

Modul implementasi dibuat pada: 10 Oktober 2026
Menggabungkan fitur *RBAC Control & CRUD Pengguna* dan *RBAC Berbasis Menu* dalam satu dokumen.